

Express Testat Code

index.js

see express.js demo

app.js

```
import express, { NextFunction, Request, Response } from 'express';
import path from 'path';
import cors from 'cors';

import { userRoutes } from './routes/user-routes';
import { accountRoutes } from './routes/account-routes';
import { transactionRoutes } from './routes/transaction-routes';
import { CONFIG } from './config';
import { expressjwt } from 'express-jwt';
import { HttpError } from './services/http-error';

export const app = express();

app.use(cors());
app.use(express.static(path.resolve('public')));
app.use(express.json());

app.use(
  expressjwt(CONFIG.jwt_validate).unless({
    path: [/\/login*/, /\/register/],
  }),
);

// app.use('/users', userRoutes);
// app.use('/accounts', accountRoutes);
app.use('/transactions', transactionRoutes);

app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof HttpError) {
    return res.status(err.status).json({ message: err.message });
  }
  if (err.name === 'UnauthorizedError') {
    return res.status(401).json({ message: 'No token / Invalid token provided' });
  }
  return res.status(400).json({ message: 'something went wrong', err });
});
```

routes/transaction-routes.ts

```
import express from 'express';
import { transactionController } from '../controller/transaction-controller';

const router = express.Router();

router.post('/', transactionController.create);
// router.get('/', transactionController.getTransactions);
// router.get('/:accountNr', transactionController.getTransactions);

export const transactionRoutes = router;
```

controller/transaction-controller.ts

```
import { Request, Response } from 'express';
import { HttpError } from '../services/http-error';
import { SchemaUtil } from '../utils/schema-util';
import { AddTransactionSchema, FindTransactionSchema, transactionService } from '../services/transaction-service';

export class TransactionController {
  public create = async (req: Request, res: Response) => {
    const auth = req.headers.authorization;
    if (!auth) { throw new HttpError(401, 'no token provided'); }
    if (!req.auth?.uuid) { throw new HttpError(401, 'Unauthorized'); }

    const data = SchemaUtil.parseOrThrow(AddTransactionSchema, req.body);

    const userUuid = req.auth.uuid;

    // import { accountService } from '../services/account-service';
    const isOwner = await accountService.isOwner(data.from, userUuid);

    (accountService code at end)

    if (!isOwner) { throw new HttpError(402, 'Incorrect user'); }
    await transactionService.create(data);
    return res.status(200).json({ success: true });
  };
};
```

services/transaction-service.ts

```
import Datastore from '@seald-io/nedb';
import { z } from 'zod';
import { accountService } from './account-service';
import { HttpError } from './http-error';

const TransactionSchema = z.object({
  from: z.number(), // can be float
  amount: z.number().int(),
  date: z.date(), to: .., total: ..
}); type Transaction = z.infer<typeof TransactionSchema>;

export const AddTransactionSchema = z.object({
  amount: z.number().int().positive(), from: .., to: ..,
}); type AddTransaction = z.infer<typeof AddTransactionSchema>;

type FindTransactionResult = {
  docs: Transaction[]; count: number; skip: number; docCount: number;
};

export const FindTransactionSchema = z.object({
  accountNr: z.coerce.number(),
  count: z.coerce.number().int().positive().optional().default(10),
  skip: z.coerce.number().int().nonnegative().optional().default(0),
}); type FindTransaction = z.infer<typeof FindTransactionSchema>;

export class TransactionService {
  private db: Datastore<Transaction>;

  constructor() { this.db = new Datastore({}); }
```

```

    async create(data: AddTransaction): Promise<void> {
        const from = await accountService.get(data.from);
        if (from.balance < data.amount) {
            throw new HttpError(400, 'Insufficient funds');
        }
        // throw error if amount < 0, sending to yourself..

        const newFromBalance = from.balance - data.amount;
        const to = await accountService.get(data.to);
        const newToBalance = to.balance + data.amount;

        const date = new Date();
        await this.db.insertAsync({ from: data.from, ... });
        await accountService.update(from.accountNr, { balance: newFromBalance });
        await accountService.update(to.accountNr, { balance: newToBalance });
    } // transactionService.create

    async find(data: FindTransaction): Promise<FindTransactionResult> {
        const filter = {
            $or: [{ from: data.accountNr }, { to: data.accountNr }],
        };

        const limit = data.count < 5 ? data.count : 5;

        let transactions: Transaction[] = await this.db
            .findAsync(filter)
            .sort({ date: -1 })
            .skip(data.skip)
            .limit(limit);

        const transactionCount = await this.db.countAsync(filter);

        return {
            docs: transactions,
            count: transactions.length,
            skip: data.skip,
            docCount: transactionCount,
        };
    } // transactionService.find
} export const transactionService = new TransactionService();

```

services/account-service.ts

```

// imports and define schemas like transaction-service.ts
export class AccountService {
    // private db with constructor like transaction-service
    // here is the file version:
    constructor() {
        this.db = new Datastore({ filename: './data/account.db', autoload: true });
        this.db.ensureIndexAsync({ fieldName: 'owner', unique: true });
        this.db.ensureIndexAsync({ fieldName: 'accountNr', unique: true });
    }

    // const isOwner = await accountService.isOwner(data.from, userUuid);
    async isOwner(accountNr: number, userId: string) {

```

```

        const account = await this.get(accountNr);
        return account.owner === userId;
    }

    // const to = await accountService.get(data.to);
    async get(accountNr: number) {
        const account = await this.db.findOneAsync({ accountNr });
        if (!account) { throw new HttpError(404, `${accountNr} not Found`); }
        return account;
    }

    // await accountService.update(to.accountNr, { balance: newToBalance });
    async update(accountNr: number, data: Update) {
        const account = await this.db.updateAsync(
            { accountNr }, { $set: data }, { multi: false, returnUpdatedDocs: true },
        );
        if (!account.affectedDocuments) { throw new HttpError(404, `${accountNr} not Found`); }
        return true;
    }
}

```